Course Project Guidelines

The course project is a central component of this course, accounting for **50%** of your final grade. It is a substantial, collaborative undertaking designed for teams of **2 to 4 students** to apply and expand upon concepts from lectures by building and deploying a **creative mobile application** using **React Native** and **Expo**. This project emphasizes mobile development technologies covered in class, including components, navigation, state management, notifications, and backend integration, while encouraging innovative and creative app ideas.

Project Deliverables

The course project consists of three major deliverables:

1. **Project Proposal** (due **Sunday, October 19, 2025, 11:59 PM**): **15% of final grade**
   - A Markdown document outlining the project's motivation, objectives, features, and tentative plan.
2. **Presentation** (delivered during Lectures 11 and 12, **Wednesday, November 19 & 26, 2025**): **10% of final grade**
   - A 5-minute in-class presentation showcasing your project's features and technical implementation, graded 5% by peer review and 5% by instructor/TAs using the same rubric.
3. **Final Project Deliverable** (due **Sunday, December 7, 2025, 11:59 PM**): **25% of final grade**
   - **Source Code**: Delivered as a public or private Git repository on GitHub. Ensure that your repository is well-organized, with clear structure and comments.
   - **Final Report**: A detailed report to ensure full reproducibility of your work, delivered as a `README.md` file in your Git repository.
   - **Video Demo**: A video demonstration of your project, lasting between 1 and 5 minutes. The demo should highlight the key features and functionality of your application. Include the video URL in the `## Video Demo` section of your final report (the `README.md` file).

Project Idea

The course project represents a more innovative and time-consuming piece of work than the course assignments, requiring teamwork to build a **creative mobile application**. It involves three key stages: **forming a team**, **writing a project proposal**, and **completing the final deliverable**. Your project **MUST** be built and deployed using React Native and Expo and **MUST** incorporate the following technologies and features to demonstrate mastery of course concepts while showcasing your creativity.

Core Technical Requirements

1 React Native and Expo Development (Required for ALL projects)

- Use **React Native** with **Expo** as the development framework, implemented in **TypeScript**.
- Implement a multi-screen app with at least four screens.
- Use core React Native components (e.g., `View`, `Text`, `TextInput`, `FlatList`) and hooks (e.g., `useState`, `useEffect`) with TypeScript typing (e.g., define types for props, state, and navigation).

2 Navigation (Required for ALL projects)

- Implement navigation using **Expo Router** or **React Navigation**.
  - For Expo Router, leverage its file-based routing (e.g., via an `app/` directory structure for screens and layouts).
  - For React Navigation, use a navigation library like `@react-navigation/native` with navigators such as Stack, Tabs, or Drawer, ensuring TypeScript support (e.g., typed navigation props and route parameters).
- Support data passing between screens (e.g., display item details on a separate screen using dynamic routes like `[id]` for Expo Router or route parameters for React Navigation).

3 State Management and Persistence (Required for ALL projects)

- Manage app state using **Context API** or **useReducer** for module-level or medium-complexity state (e.g., theme, user preferences, small data sets).
  - **Note**: Context is primarily a dependency injection mechanism. While it works for state management in moderate cases, it may not scale efficiently for very large or frequently updated global state.
- Optionally, teams may choose to use **Redux Toolkit** for app-wide or complex state management, either supplementing or replacing Context/useReducer in parts of the app.
- Persist data locally using **React Native Async Storage** to retain state across app restarts (e.g., user inputs, preferences).

4 Notifications (Required for ALL projects)

- Implement **Expo Notifications** for at least one local notification (e.g., a daily reminder to interact with the app).
- Handle notification scheduling and user interactions (e.g., responding to notification taps).

5 Backend Integration (Required for ALL projects)

- Integrate your app with a **backend service or API**. Options include:
  - A REST API (mock, public, or custom-built) using fetch or Axios
  - A Backend-as-a-Service (BaaS) platform (e.g., Supabase, Firebase)
- Fetch and display live data from an external source (not just local static JSON), updating the UI dynamically (e.g., showing API-fetched items in a list).
- Handle responses and errors gracefully (e.g., loading states, error messages, retries when appropriate).

- Support API-driven navigation (e.g., using Expo Router's search params or React Navigation's route parameters for dynamic routing).

6 Deployment (Required for ALL projects)

- Build and deploy the app using **Expo EAS Build**.
- Must provide a **testable Android build** in one of the following formats:
  - APK file (recommended)
    - Submit a downloadable link (e.g., Sync.com, Google Drive, Dropbox), or
    - If the APK is under 100 MB, you may upload it directly to your GitHub repository
  - EAS Build link (QR code optional, as it points to the same link)
- Optional: You may also include an iOS build (IPA or TestFlight), but the Android build will be used for grading.

Advanced Features (Must implement at least two)

- **User Authentication**: Implement login/logout functionality (e.g., email/password or social provider login). Ensure proper handling of authentication state and secure storage of tokens where applicable. Options include:
  - OAuth / Social login via services like GitHub, Google, or Facebook (e.g., using Expo AuthSession ↗)
  - Backend-managed authentication (e.g., Firebase Authentication ↗, Supabase Auth ↗, Appwrite Authentication ↗)
  - Any other method that allows secure user authentication without storing sensitive credentials directly in the app
- **Real-Time Updates**: Add real-time functionality to your app (e.g., live chat or task updates). Options include:
  - WebSockets (e.g., Socket.IO, native WebSocket API)
  - Backend-as-a-Service (BaaS) with real-time support (e.g., Supabase Realtime ↗, Firebase Realtime Database ↗)
- **Mobile Sensors or Device APIs**: Integrate at least one mobile-specific capability using Expo or React Native APIs. Handle permissions properly, requesting and responding to user access for camera, motion data, or location. Options include:
  - Sensors: `expo-sensors` (e.g., pedometer, accelerometer, gyroscope)
  - Camera: `expo-camera` (e.g., photo capture, QR code scanning)
  - Location: `expo-location` (e.g., maps, nearby points of interest, geofencing)
- **Offline Support**: Enable offline functionality by caching data (e.g., view saved content offline). Options include:
  - React Native Async Storage ↗
  - Expo SQLite ↗
  - WatermelonDB ↗ or other local database solutions

- **Custom Animations**: Add interactive UI animations to enhance user experience (e.g., transitions, progress indicators). Options include:
  - React Native Reanimated ↗
  - React Native's Animated API ↗ .
  - DotLottie React Native ↗
- **Gamification**: Incorporate engagement features like badges, streaks, or progress tracking (e.g., rewarding users for completing tasks or reaching milestones).
- **Integration with External Services**: Connect to external APIs (e.g., weather, news, or calendar APIs for dynamic content).
- **Accessibility Enhancements**: Implement accessibility features, such as screen reader support (e.g., VoiceOver, TalkBack) or high-contrast mode, using React Native's accessibility APIs ↗ (e.g., `accessibilityLabel`, `accessibilityRole`). Test with assistive technologies to ensure usability.
- **Push Notifications**: Implement push notifications to notify users of important events or updates (e.g., event reminders, task updates, or live alerts). Ensure permission handling and user opt-in/opt-out flows are correctly implemented. Options include:
  - Expo Push Notifications ↗
  - Firebase Cloud Messaging ↗
  - OneSignal, Pusher Beams, AWS SNS, or other standard platforms
- **Social Sharing**: Enable content sharing via Expo Sharing ↗ (e.g., share app content to social media or messaging apps).

Note on Project Requirements and Scope

> ! **Meeting the Core Technical Requirements is mandatory — no exceptions.** Projects must use specified tools (e.g., React Native, Expo) to align with course content, ensure fair grading and peer review, and keep complexity manageable. Alternative frameworks (e.g., Flutter, native iOS/Android) are not permitted. However, students are encouraged to explore advanced features and creative app ideas within the approved tools.

Your project **MUST** implement all core technologies and at least two advanced features. To ensure an achievable project within the ~2-month timeline, avoid these pitfalls:

- **Too simple**: A basic CRUD app without notifications or backend integration shows insufficient mastery.
- **Too complex**: Machine learning or complex backend systems are hard to complete effectively.
- **Too broad**: A full-scale platform (e.g., a social media app) risks shallow implementation.

The **goal** is to develop a **well-scoped, creatively implemented mobile application** that effectively showcases your understanding of mobile development principles using the required technologies. A successful project should have a **clear purpose, achievable scope, and polished implementation of core features**, with creative elements that reflect the course's focus on innovative app design.

Remember: A well-executed, focused project is far more valuable than an overly ambitious one that cannot be completed properly within the course timeline. While creativity and innovation are encouraged, please keep in mind that **this is a course project with specific learning objectives to achieve.**

Example Project Ideas

Below are some inspiring project ideas to guide your team. Smaller teams (2 members) may implement a subset of features, while larger teams (3-4 members) should aim for more features or added complexity. **All projects must fulfill the Core Technical Requirements**. The listed features illustrate potential scope rather than strict expectations. Teams are also encouraged to propose unique project ideas tailored to their interests and expertise, provided the scope is realistic and achievable within the project timeline.

1. **Collaborative Task Manager**

   A mobile app for teams to manage tasks with real-time collaboration and device integration.

   - **Key Features**:
     - Create, edit, and delete tasks (title, due date, priority, etc.) in a FlatList.
     - Navigate between Home, Add Task, and Task Details screens.
     - Persist tasks using React Native Async Storage and Context API.
     - Schedule task reminders with Expo Notifications.
     - Fetch task categories from a mock REST API.
     - Advanced: Use Expo Camera to attach photos to tasks.
     - Advanced: Real-time task updates using WebSockets.
     - Advanced: User authentication with Firebase for team accounts.

---

2. **Recipe Organizer**

   A mobile app for users to save and organize recipes with external data integration.

   - **Key Features**:
     - Add and display recipes (name, ingredients, instructions, etc.) in a FlatList.
     - Navigate between Home, Add Recipe, Recipe Details, and an additional user-specific content screen (e.g., collections or saved recipes).
     - Persist recipes using React Native Async Storage and useReducer.
     - Send cooking reminders with Expo Notifications.
     - Fetch recipe suggestions from a public food API.
     - Advanced: Use Expo Location to suggest recipes based on local markets, or Expo Camera to capture recipe images
     - Advanced: Social sharing of recipes via Expo Sharing.
     - Advanced: Accessibility Enhancements with screen reader support.

3. **Event Planner**

   A mobile app for users to manage events with reminders and location-based features.

   - **Key Features**:
     - Log events (title, date, location, etc.) with a form and list view.
     - Navigate between Home, Add Event, Event Details, and an additional user-specific content screen (e.g., event collections or calendar).
     - Persist events using React Native Async Storage and Context API.
     - Schedule event reminders with Expo Notifications.
     - Fetch event categories from a mock API.
     - Advanced: Use Expo Location to mark event locations on a map, or Expo Camera to capture event posters.
     - Advanced: Push notifications via Firebase for event updates.
     - Advanced: Accessibility enhancements for screen readers.

---

4. **Study Planner**

   A mobile app for students to plan study sessions with device integration.

   - **Key Features**:
     - Create and view study sessions (subject, duration, notes) in a FlatList.
     - Navigate between Home, Add Session, Session Details, and Schedule View screens.
     - Persist sessions using React Native Async Storage and useReducer.
     - Send study reminders with Expo Notifications.
     - Fetch study tips from a public API.
     - Advanced: Use Expo Pedometer to suggest breaks after long study sessions.
     - Advanced: Gamification with badges for study milestones.
     - Advanced: Custom animations for progress tracking.

---

5. **Local Guide**

   A mobile app for discovering and saving local points of interest (e.g., cafes, parks).

   - **Key Features**:
     - Add and view points of interest (name, location, description, etc.) in a FlatList.
     - Navigate between Home, Add Place, Place Details, and an additional user-specific content screen (e.g., favorites or recommendations).

- Persist places using React Native Async Storage and Context API.
- Send exploration reminders with Expo Notifications.
- Fetch place categories from a public API (e.g., Google Places).
- Advanced: Use Expo Location to show nearby places, or Expo Camera to capture photos of places.
- Advanced: Offline Support for viewing saved places without internet.
- Advanced: Integration with External Services (e.g., weather API for visit recommendations).

Clarification on Team Size and Grading

Grading is consistent for all teams (2-4 students), with expectations scaled by team size. All teams must implement core technical requirements and at least two advanced features. Smaller teams (2 members) may use simpler implementations, while larger teams (3-4 members) should add complexity, as shown in Example Project Ideas.

Rubrics for the Project Proposal, Presentation, and Final Deliverable focus on quality and requirements, not team size. The Project Completion rubric adjusts code expectations: 800+ lines per member (2-member teams), 600+ (3-member teams), 500+ (4-member teams). Individual contributions are verified via GitHub commits to ensure fairness. More members mean more coordination, which balances workload across team sizes.

Project Proposal

The project proposal should be submitted as a single file in the form of a Markdown document (with the `.md` suffix in the filename), with a maximum length of 2000 words. Other formats (such as Microsoft Word or Adobe PDF) will not be accepted, as Markdown is the industry standard for technical documentation in software development. The project proposal should include the following three sections of the project, described clearly and concisely:

Required Sections

1. Motivation

- Identify the problem or need your project addresses
- Explain why the project is worth pursuing
- Describe target users
- Optional: Discuss existing solutions and their limitations

2. Objective and Key Features

- Clear statement of project objectives
- Detailed description of core features, including:

    - Navigation structure (e.g., screens, file-based routes, and layouts for Expo Router, or navigators like Stack, Tabs, or Drawer for React Navigation with TypeScript typing)
    - State management and persistence approach
    - Notification setup
    - Backend integration details
    - Deployment plan with Expo EAS Build
    - Planned advanced features (at least two)
- Explanation of how these features fulfill the course project requirements
- Discussion of project scope and feasibility within the timeframe

3. Tentative Plan

- Describe how your team plans to achieve the project objective in a matter of weeks, with clear descriptions of responsibilities for each team member
- No need to include milestones and tentative dates, as the duration of the project is short

Marking Rubrics

Motivation: 30% (out of 10 Points)

- **10 Points**: The motivation is sufficiently convincing, with a clear problem statement and well-defined target users. The proposal demonstrates thoughtful consideration of why this project is worth pursuing and how it benefits its intended users.
- **6 Points**: The motivation is present but lacks conviction or clarity. The problem statement or target users are vaguely defined, making it difficult to understand the project's value.
- **0 Point**: The motivation section is missing or completely irrelevant to the project scope.

Objective and Key Features: 50% (out of 10 Points)

- **10 Points**: The objectives are clearly defined with specific technical features that properly utilize course technologies. The scope is realistic for the team size and project timeline, demonstrating a good balance between innovation and feasibility.
- **6 Points**: The objectives and features are present but lack clarity or completeness. Some required technologies may be underutilized, or the scope may need adjustment to be more realistic for the timeline.
- **0 Point**: The objectives and features section is missing or fails to address the basic course project requirements.

Tentative Plan: 20% (out of 10 Points)

- **10 Points**: The proposed plan is concise and clear, includes responsibilities for each team member, and a casual reader can be convinced that the project can be reasonably completed by the project due data.
- **6 Points**: The proposed plan has been included, but not clear to a casual reader.
- **0 Point**: The proposed plan is missing or incomprehensible.

Submission

Submit a single Markdown document to the assignment labeled **Project Proposal** in the Quercus course website ↗ by **Sunday, October 19, 2025, 11:59 PM**. Each member of the team should make their own submission, but obviously all members in the same team should submit the same document.

> **Note on Project Changes Post-Proposal**
>
> You are allowed to modify your project idea, features, or scope after submitting the proposal (e.g., based on challenges, or new insights). We will not enforce consistency between the proposal and your final deliverables — each will be graded independently using the provided marking rubrics.

Presentation

Each team will deliver a **5-minute presentation** during Lecture 11 (November 19, 2025) or Lecture 12 (November 26, 2025) to showcase the project's features and technical implementation.

Presentation slots have been randomly assigned and can be viewed here:

- November 19 Slots
- November 26 Slots

Teams may designate 1–2 members to present, but all members must attend both sessions to provide peer feedback. Exceptions are granted only to part-time MEng students with unavoidable work conflicts.

Submission

- Submit presentation slides (or outline) individually via Quercus ↗ by **Monday, November 17, 2025, 11:59 PM**.
- All team members must upload the same file to ensure fairness.
- No major changes are allowed after the deadline.

Content

- **Core Requirements (Mandatory)**: Demonstrate all core technical requirements (except deployment) in a local environment. Highlight TypeScript usage in key features (e.g., typed components, navigation, or APIs) via slides or verbal explanation. Outline deployment plans.
- **Advanced Features Plan**: Present a feasible plan for at least two advanced features, including a simplified demo or mockup if partially implemented, with a completion timeline by December 7, 2025.
- Highlight the app's creative purpose, design, key features, and user flow.

Expectations

- **Core Requirements**: By November 17, 2025, all core requirements (except deployment) must be functional and demo-ready in a local environment (e.g., Expo Go), including:

- React Native/Expo with TypeScript
- Navigation (Expo Router or React Navigation)
- State management and persistence
- Notifications
- Backend integration

  Teams may refine or enhance these features before the final deliverable deadline (December 7, 2025).

- **Advanced Features**: May be in progress, but a clear, realistic completion plan is required.
- **Demo**: Use a live demo to showcase all core features, briefly noting TypeScript usage (e.g., in slides or verbally). Optionally include code snippets for clarity.

Grading

- Worth 10% of final grade (5% peer, 5% instructor/TAs), evaluated using the Presentation Rubric.
- Peer scores are normalized across days for fairness.

Final Project Deliverable

The final project deliverable should be submitted as a URL to a public or private GitHub repository. If your repository is private, add the instructor and TAs (GitHub usernames: `cying17`, `Yiningww`, and `YirenZzz`) as collaborators so that it can be read. Your repository must contain:

1. A final report (`README.md`)
2. Complete source code
3. A video demo
4. Deployment details for your required Android build (APK file, EAS Build link, or downloadable APK link). Optional iOS builds (IPA or TestFlight) may also be included but are not required.

Final Report

A `README.md` file that contains the final report, in the form of a Markdown document ↗ of no more than 5000 words in total length [1] [2]. If you wish to include images (such as screenshots) in the final report, make sure that they are visible when the instructor and TAs visit your GitHub repository with a web browser. The final report should include the following logistical and technical aspects of the project, described clearly and concisely:

- **Team Information**: The names, student numbers, and preferred email addresses of all team members. Ensure these email addresses are active as they may be used for clarification requests.
- **Motivation**: What motivated your team to spend time on this project? Describe the problem you're solving and its significance.
- **Objectives**: What are the objectives of this project? Explain what you aimed to achieve through this implementation.
- **Technical Stack**: What technologies were used in the project? Include the chosen navigation (Expo Router or React Navigation with TypeScript), state management approach (Context API, useReducer, or Redux Toolkit),

and other key technologies.

- **Features**: What are the main features offered by your application? Describe how they fulfill the course project requirements and achieve your project objectives.
- **User Guide**: How does a user interact with your application? Provide clear instructions for using each main feature, supported with screenshots where appropriate.
- **Development Guide**: What are the steps to set up the development environment? Include detailed instructions for environment, dependencies, and local testing.
- **Deployment Information**: Include the build details for your required Android build (APK file, EAS Build link, or downloadable APK link). Optional: If available, you may also include an iOS build (IPA or TestFlight link), but the Android build will be used for grading.
- **Individual Contributions**: What were the specific contributions made by each team member? This should align with the Git commit history.
- **Lessons Learned and Concluding Remarks**: Share insights gained during the development process and any final thoughts about the project experience, if any.

Source Code

Your GitHub repository must contain all code required to build and run the project, organized in a clear, logical directory structure. Required components include:

- Application code (React Native with Expo, implemented in **TypeScript**, using TSX)
- Navigation configuration (Expo Router with TypeScript types for navigation and auto-generated route types from file structure, or React Navigation with TypeScript types for navigators and route parameters)
- State management and persistence setup (Context API/useReducer/Redux Toolkit with TypeScript types, React Native Async Storage)
- Local notification setup using Expo Notifications (with proper TypeScript types and permission handling)
- API integration code (fetch/Axios, with TypeScript types for API responses)
- Expo EAS Build configuration

Include detailed setup and runtime instructions in the `README.md`'s Development Guide for local execution.

Your code should follow consistent formatting, use TypeScript type annotations for all components, hooks, and APIs, and include appropriate comments for complex logic. Consider including a `.gitignore` file to exclude unnecessary files and dependencies from version control.

> ⓘ If your project requires sensitive credentials (e.g., API keys) for execution, submit them in a password-protected `.zip` or `.tar.gz` file via email to TA Yiren Zhao: yiren.zhao@mail.utoronto.ca
>
> Send the password in a separate email to the TA. Both emails must be sent **by the final deliverable deadline**. Each team only needs to complete this step once.
>
> In your Development Guide, clearly state "Credentials sent to TA".

Video Demo

You need to include a video demo, lasting between 1 and 5 minutes, showing:

- Key features in action
- User flow through the app
- Technical highlights
- The app running from the deployed build (e.g., APK, IPA) on an emulator or device

The video's URL must be included in the `## Video Demo` section of the `README.md`. Host the video on a platform like YouTube, Dropbox, or Google Drive, ensuring access for the instructor and TAs. If under 100 MB, the video may be included directly in the GitHub repository.

Marking Rubrics

Technical Implementation: 30% (out of 20 Points)

To be marked by reading the final report `README.md`, reviewing source code and testing the application functionality. Evaluation includes the use of TypeScript type annotations for components, hooks, navigation, and APIs.

- **20 Points**: Complete and correct implementation of all required technologies, including consistent and correct TypeScript usage
- **15 Points**: Implementation has minor issues in one or two areas, including minor TypeScript inconsistencies
- **10 Points**: Basic implementation with several issues, including inconsistent TypeScript usage
- **5 Points**: Major issues in implementation, including significant TypeScript errors or omissions
- **0 Points**: Missing critical technical components or TypeScript usage

Project Completion: 30% (out of 20 Points)

To be marked by reading the final report `README.md`, reviewing source code and watching the video demo.

- **20 Points**: All proposed features working correctly with clear user flow. Meets minimum lines of meaningful code (excluding comments, `node_modules`, generated files, etc.) per member: 800+ (2 members), 600+ (3 members), or 500+ (4 members).
- **15 Points**: Most features working as intended. Lines of meaningful code per member: 600-800 (2 members), 450-600 (3 members), or 350-500 (4 members).
- **10 Points**: Basic features implemented. Lines of meaningful code per member: 400-600 (2 members), 300-450 (3 members), or 250-350 (4 members).
- **5 Points**: Features are significantly unfinished, or lines of meaningful code fall below the minimum threshold for the team size.
- **0 Points**: Minimal working functionality or insufficient code contribution.

> Lines of code (LOC) will be counted using cloc ↗, excluding comments, `node_modules`, generated files, and non-source files such as configuration and dependency files.

Documentation and Code Quality: 20% (out of 20 Points)

To be marked by reading the final report `README.md` and reviewing code organization.

- **20 Points**: Comprehensive and well-structured `README.md` with clear setup instructions, a well-organized codebase, consistent coding style, and regular, meaningful Git commits
- **15 Points**: Good documentation with minor gaps in clarity, structure, or consistency
- **10 Points**: Basic documentation is present but lacks essential details, inconsistent code organization
- **5 Points**: Incomplete or unclear documentation or messy code
- **0 Points**: Missing documentation or chaotic code

Individual Contributions: 20% (out of 20 Points)

Individual marks will be assigned to each team member based on reading the final report `README.md` and reading the commit messages in the commit history in the GitHub repository.

- **20 Points**: The team member has made a fair amount of contributions to the project, without these contributions the project cannot be successfully completed on time.
- **15 Points**: The team member has made less than a fair amount of contributions to the project, *or* without these contributions the project can still be successfully completed on time.
- **5 Points**: The team member has made less than a fair amount of contributions to the project, *and* without these contributions the project can still be successfully completed on time.
- **0 Point**: The team member has not made any contributions to the project.

Bonus Points

Each of the following achievements grants 3% bonus points to the final project deliverable grade:

- **Notable Innovation in Implementation**: Implement a highly creative feature that exceeds course project requirements by using course technologies in a novel way (e.g., a unique gamification system combining multiple Expo APIs, or innovative use of Expo Camera for real-time analysis).
- **Exceptional User Experience**: Deliver a highly polished UI/UX that goes beyond basic advanced feature implementation. Examples include complex custom animations (e.g., multi-step transitions via React Native Reanimated), comprehensive accessibility support (e.g., extensive screen reader testing with React Native accessibility APIs), or a visually cohesive design with professional-grade styling and user flow.
- **High-Quality Open Source Project**: Create a project that is usable by external developers, including:
  - Clear contribution guidelines (e.g., how to submit issues or pull requests)
  - Well-documented API/components (e.g., TypeScript interfaces or function documentation)
  - Detailed installation and setup instructions for running the app
  - An MIT or similar open-source license
  - A professional README structure with sections for installation, usage, and contribution guidelines

Note: Bonus points are awarded for exceeding course expectations, with a maximum of 9% extra credit added to the final project deliverable grade (25% of final grade). Comprehensive accessibility efforts (e.g., extensive

support for screen readers, high-contrast modes, or thorough testing with assistive technologies) are particularly valued for their impact on inclusive design.

Submission

Submit a single URL — the URL to your team's GitHub repository — to the assignment labeled **Final Project Deliverable** in the Quercus course website ↗. Each member of the team should make their own submission, but obviously all members in the same team should submit the same URL.

> ⚠ The final project deliverable is due **Sunday, December 7, 2025, 11:59 PM** Eastern Time. Late submissions are **NOT** accepted. For private GitHub repositories, add the instructor and TAs (GitHub usernames: `cying17`, `Yiningww`, `YirenZzz`) as collaborators before the deadline.

Tips and Suggestions

Using GitHub for Project Management and Effective Collaboration

Effective teamwork depends on efficient communication and collaboration, often more so than completing individual tasks. To streamline your team's workflow, I highly recommend using **GitHub** as your central collaboration hub. Treat your GitHub repository as your team's central workspace — it's not just for code storage but also a powerful tool for organizing and managing your entire project. Here's how to leverage its full functionality:

- **Commit Frequently and Meaningfully**: Every change to the code should be committed to the repository immediately, accompanied by a **complete, concise, and meaningful commit message**. This helps everyone stay on the same page and track progress over time.
- **Use Branches and Pull Requests**: Create a new branch for each feature or task. Once the feature is complete, submit a pull request for review. This ensures code quality, facilitates collaboration, and makes it easier to integrate changes.
- **Use GitHub Issues for Task Management**: Create GitHub Issues to maintain a "to-do" list of outstanding tasks. Assign issues to team members, add labels for priority or category, and close them once completed. This keeps your workflow organized and transparent.
- **Leverage GitHub Discussions for Communication**: Use GitHub Discussions to document all team conversations, decisions, and brainstorming sessions. This ensures that nothing gets lost, and you can always refer back to previous discussions if needed.
- **Document Everything in the GitHub Wiki**: Use the GitHub Wiki as your project journal to record major decisions, challenges, and solutions. This creates a valuable reference for your team and anyone who might review your project in the future.

Check this page for a step-by-step guide on using GitHub to manage tasks and collaborate effectively.

1. There is no minimum length requirement for the final report in `README.md`. Other formats (such as Microsoft Word or Adobe PDF) will not be accepted, as Markdown is the standard documentation format in software development. ↩

2. You may reuse relevant content from your project proposal where appropriate. ↩

Last updated on September 1, 2025